School of Computer Science and Artificial Intelligence

Lab Assignment # 02

Course Title	**: AI Assistant Coding**
Name of Student	**: M. Sai Pallavi**
Enrollment No.	**: 2303A54063**
Batch No.	**48**

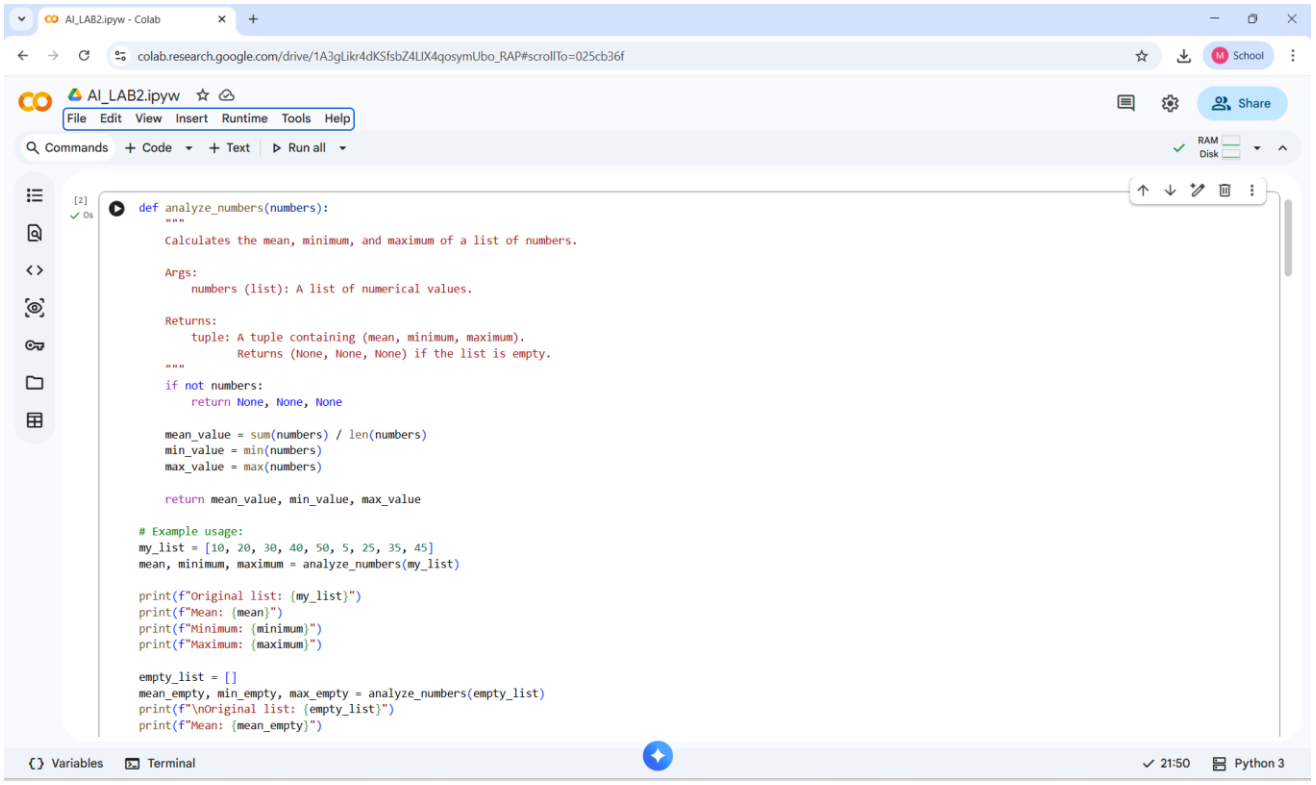
Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI

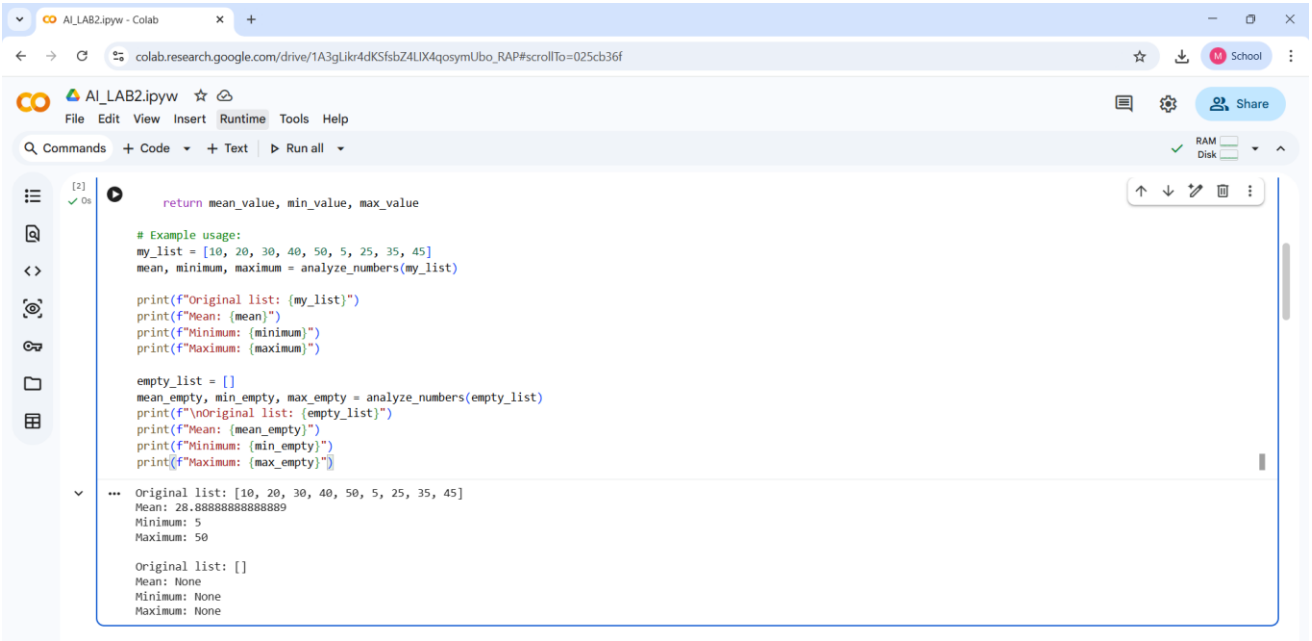
Task 1: Statistical Summary for Survey Data

❖ ***Scenario: You are a data analyst intern working with survey responses stored as numerical lists.***

• **Prompt used:** Write a Python function that takes a list of numbers and returns the mean, minimum, and maximum values.

• **Screenshot of Generated Code:**





Short Explanation of Logic:

The mean represents the average of a set of numbers. It is obtained by adding all the values in the list and dividing the sum by the total number of elements. The minimum value is the smallest number present in the list, and the maximum value is the largest number present in the list. These values are calculated using Python's built-in min() and max() functions. The function accepts a list of numerical values as input, computes the mean, minimum.

Task 2: Armstrong Number – AI Comparison

❖ ***Scenario: You are evaluating AI tools for numeric validation logic.***

• **Prompt used (Gemini):** "Write a Python program to check whether a given number is an Armstrong number."
• **Prompt used (Copilot):** "Write a Python program to check whether a given number is an Armstrong number."

• **Screenshot of Generated Code:**

AI_LAB2.ipyw - Colab

colab.research.google.com/drive/1A3gLikr4dKSfsbZ4LIX4qosymUbo_RAP#scrollTo=025cb36f

AI_LAB2.ipyw

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
[3]
✓ 0s
        num_digits = len(num_str)
        sum_of_powers = 0

        for digit_char in num_str:
            digit = int(digit_char)
            sum_of_powers += digit ** num_digits

        return sum_of_powers == number

    # Example usage:
    print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}") # Expected: True (1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153)
    print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}") # Expected: True (3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370)
    print(f"Is 9474 an Armstrong number? {is_armstrong_number(9474)}") # Expected: True (9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474)
    print(f"Is 10 an Armstrong number? {is_armstrong_number(10)}")   # Expected: False
    print(f"Is 1 an Armstrong number? {is_armstrong_number(1)}")     # Expected: True (1^1 = 1)
```

```
Is 153 an Armstrong number? True
Is 370 an Armstrong number? True
Is 9474 an Armstrong number? True
Is 10 an Armstrong number? False
Is 1 an Armstrong number? True
```

AI_LAB2.ipyw - Colab

colab.research.google.com/drive/1A3gLikr4dKSfsbZ4LIX4qosymUbo_RAP#scrollTo=025cb36f

AI_LAB2.ipyw

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

Git Hub Co-pilot code

```
[6]
✓ 0s
    def is_armstrong(number: int) -> bool:
        """
        Check if a given number is an Armstrong number.

        An Armstrong number of 'n' digits is a number such that
        the sum of its digits raised to the power 'n' equals the number itself.
        """
        # Convert number to string to easily iterate over digits
        digits = str(number)
        num_digits = len(digits)

        # Compute sum of each digit raised to the power of num_digits
        total = sum(int(digit) ** num_digits for digit in digits)

        return total == number


    # Example usage
    if __name__ == "__main__":
        test_numbers = [153, 9474, 123, 407]
        for num in test_numbers:
            if is_armstrong(num):
                print(f"{num} is an Armstrong number.")
            else:
                print(f"{num} is NOT an Armstrong number.")
```

```
153 is an Armstrong number.
9474 is an Armstrong number.
123 is NOT an Armstrong number.
407 is an Armstrong number.
```

Variables Terminal

21:50 Python 3

AI_LAB2.ipyw - Colab

colab.research.google.com/drive/1A3gLikr4dKSfsbZ4LIX4qosymUbo_RAP#scrollTo=025cb36f

AI_LAB2.ipyw

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

```
[3]
✓ 0s
    def is_armstrong_number(number):
        """
        Checks if a number is an Armstrong number.

        An Armstrong number is a number that is the sum of its own digits
        each raised to the power of the number of digits.

        Args:
            number (int): The integer to check.

        Returns:
            bool: True if the number is an Armstrong number, False otherwise.
        """
        if not isinstance(number, int) or number < 0:
            raise ValueError("Input must be a non-negative integer.")

        # Convert number to string to get its digits and count them
        num_str = str(number)
        num_digits = len(num_str)
        sum_of_powers = 0

        for digit_char in num_str:
            digit = int(digit_char)
            sum_of_powers += digit ** num_digits

        return sum_of_powers == number
```

- **Side-by-Side Comparison Table:**

Feature	Google Gemini	GitHub Copilot
Code structure	Uses a separate function	Written directly in main logic
Readability	Very clear and modular	Simple and straightforward
Logic clarity	Easy to understand with function	Easy but less modular
Beginner friendly	High	Medium
Explanation support	Provides explanation along with code	No explanation, only code
Best use case	Learning and documentation	Fast coding inside editor

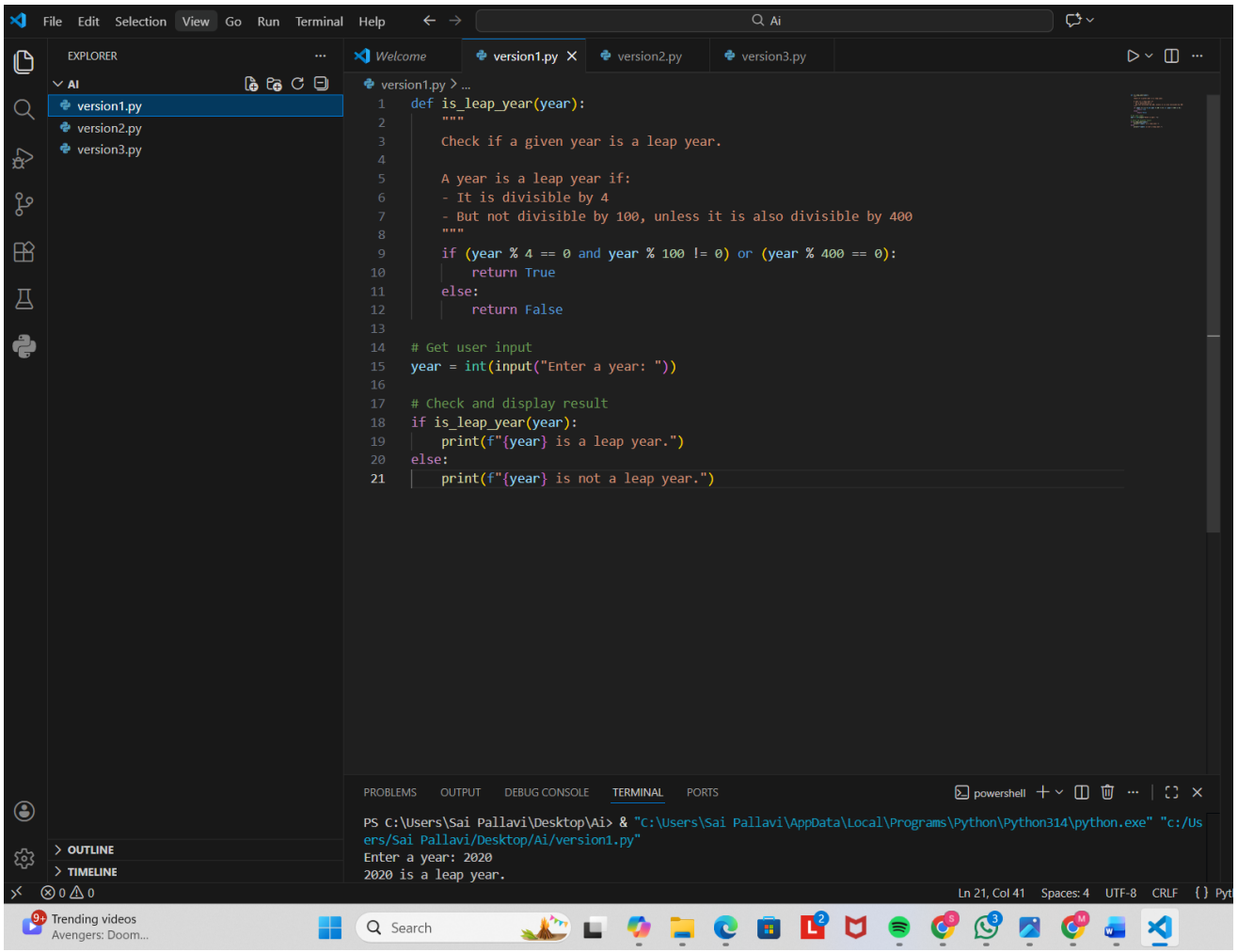
Task 3: Leap Year Validation Using Cursor AI

❖ ***Scenario: You are validating a calendar module for a backend system.***

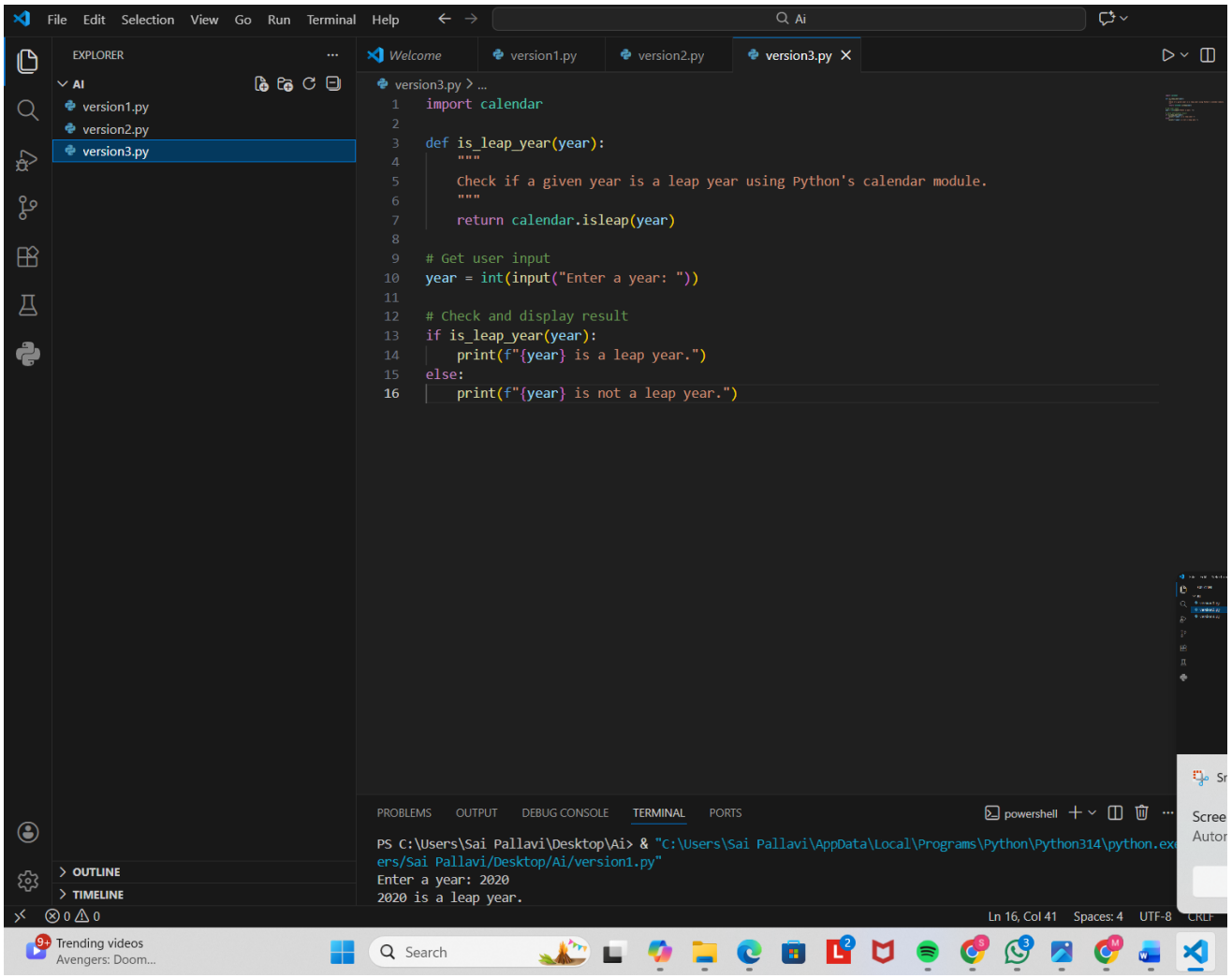
• **Prompt 1:** "Write a Python program to check whether a given year is a leap year."

• **Prompt 2:** "Write an optimized Python program with proper conditions and comments to check whether a year is a leap year."

• **Screenshot of Generated Code(V1):**



• Screenshot of Generated Code(V2):



• **Sample Input:** Enter a year: 2026
• **Sample Output:** 2026 is not a leap year.

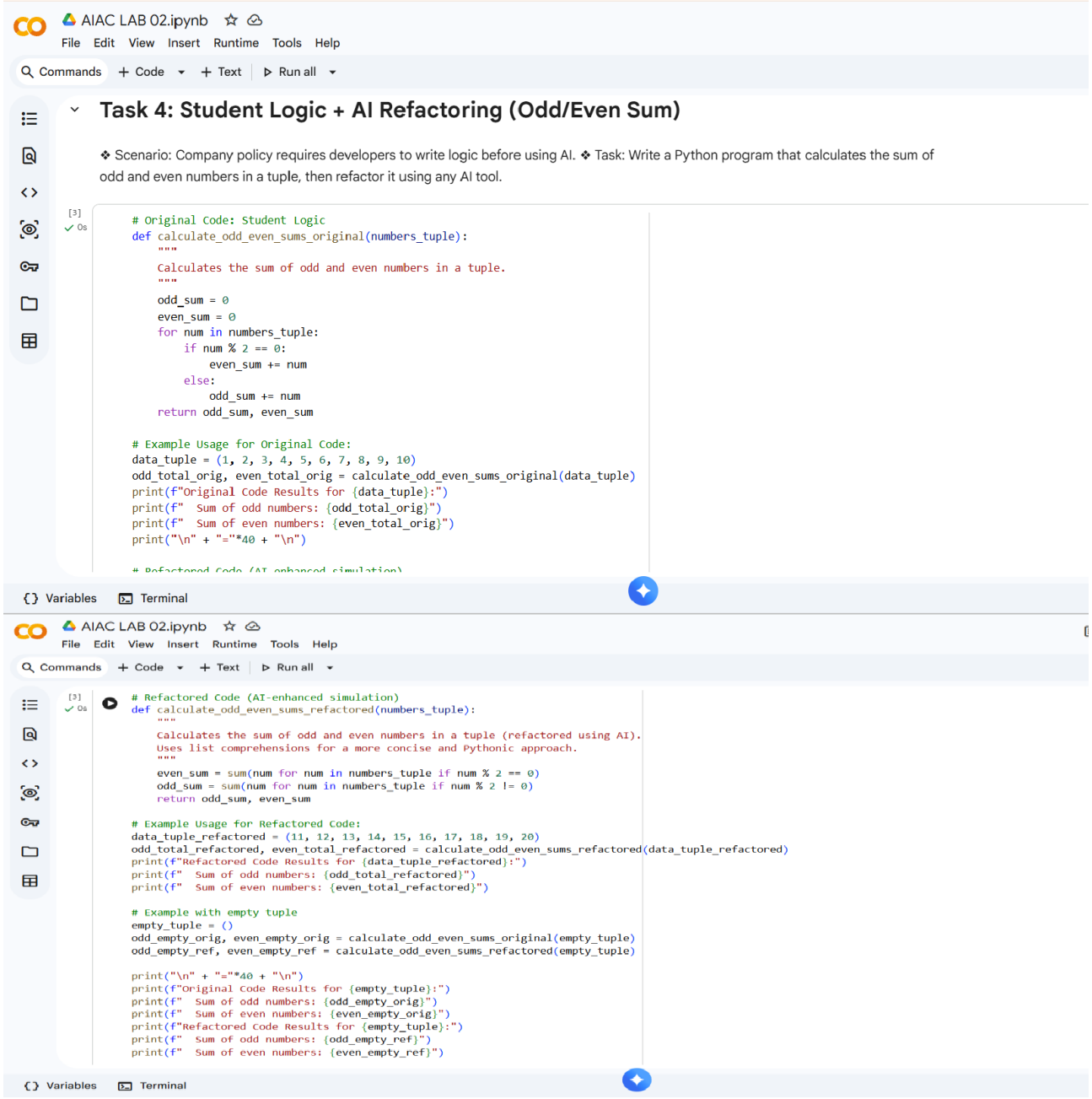
• **Short Explanation of Logic:**
version1.py manually checks leap year rules using modulo arithmetic, making the logic explicit and educational with no dependencies. version3.py uses Python's built-in calendar.isleap(), which is shorter, cleaner, and production-ready but depends on the standard library. Both are fast and correct: the first emphasizes learning the rules, while the second emphasizes simplicity.

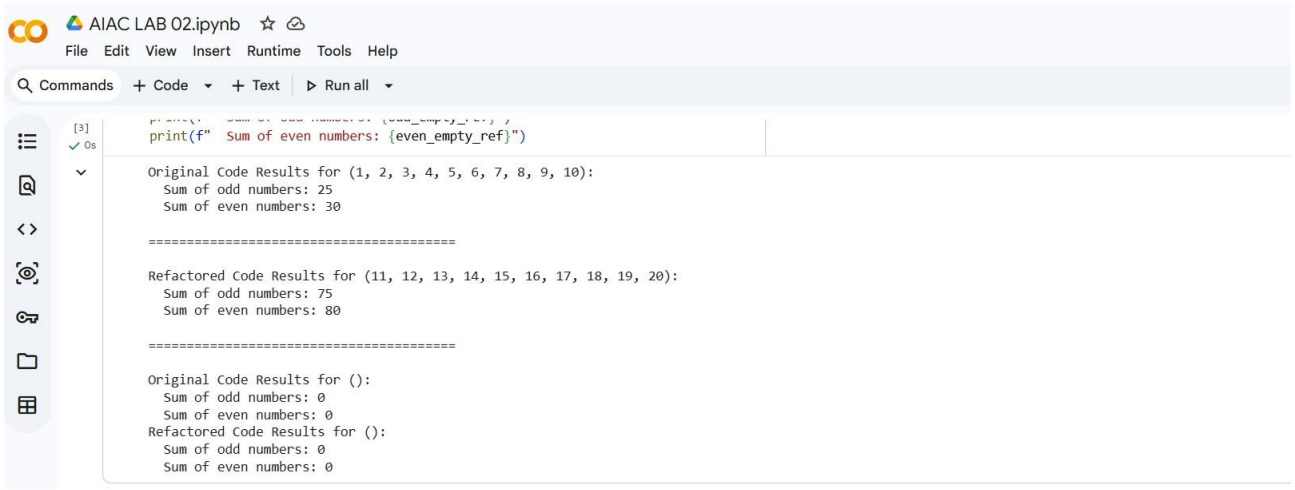
Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ ***Scenario: Company policy requires developers to write logic before using AI.***

• **Prompt used:** "Refactor this Python code to improve readability and efficiency."

• **Screenshot of Generated Code**





• Explanation

write Explanation of improvements for original code and refracted code

Empty cell

Certainly! Here's a summary of the improvements between the original and refactored code for calculating the sum of odd and even numbers:

Original Code (`calculate_odd_even_sums_original`):

- **Explicit Loop and Conditionals:** Uses a traditional `for` loop with `if-else` statements to iterate and check each number, explicitly managing `odd_sum` and `even_sum` variables.
- **Readability (Beginner):** Easy to understand for those new to programming due to its step-by-step nature.
- **Verbosity:** More lines of code are required to achieve the desired outcome.

Refactored Code (`calculate_odd_even_sums_refactored`):

- **Conciseness & Pythonic Style:** Achieves the same result in significantly fewer lines of code by using generator expressions within the built-in `sum()` function. This is considered a more idiomatic and expressive Python approach.
- **Readability (Experienced Pythonista):** For experienced Python developers, this version is often more readable as it clearly states *what* is being aggregated (sum of even numbers, sum of odd numbers) rather than *how* the iteration and conditional logic are handled.